GUÍA DEL PROYECTO

Reproductor de música

Un cliente de YouTube Music en Rust y Qt, explicado archivo por archivo



Para quien sabe programar pero empieza con Rust y QML.
Arquitectura, código, decisiones y recetas para seguir desarrollándolo.

Compilar: `latexmk -xelatex guia.tex` (carpeta `docs/guia`)

Índice general

1. Cómo leer esta guía **1**

1.1. El recorrido . . . 1

1.2. Convenciones . . . 1

1.3. Documentos hermanos . . . 2

I Empezar con buen pie **3**

2. Rust desde cero, lo justo para este proyecto **4**

2.1. Un proyecto de prácticas . . . 4

2.2. Variables, tipos y funciones . . . 4

2.3. Control de flujo . . . 6

2.4. Structs: datos con nombre . . . 6

2.4.1. Métodos . . . 7

2.5. Enums y `match` . . . 7

2.6. Sin null ni excepciones: `Option` y `Result` . . . 8

2.6.1. El operador `?` . . . 9

2.7. Propiedad y préstamos . . . 9

2.7.1. Mover . . . 10

2.7.2. Prestar . . . 10

2.7.3. Clonar . . . 10

2.7.4. `Copy` . . . 10

2.7.5. Cómo resolver los errores de propiedad . . . 10

2.7.6. `String` frente a `&str` . . . 11

2.7.7. Vidas (*lifetimes*), solo para reconocerlas . . . 11

2.8. Colecciones . . . 11

2.9. Clausuras e iteradores . . . 12

2.9.1. `move` en clausuras . . . 12

2.10. Traits: comportamientos compartidos . . . 12

2.11. Módulos y visibilidad . . . 13

2.12. Macros . . . 13

2.13. Hilos y estado compartido . . . 14

2.13.1. Lanzar un hilo 14
2.13.2. Compartir datos: `Mutex` 14
2.13.3. Otros mecanismos que verás 14
2.13.4. El patrón del proyecto: trabajar en otro hilo y volver al de Qt 15
2.14. Llamar a C y C++: `extern` y `unsafe` 15
2.15. Atributos condicionales y pruebas 15
2.16. Leer los errores del compilador 16
2.17. Resumen y siguientes pasos 17

II Cómo está hecho **18**

III Por qué está así **19**

IV Desarrollar por tu cuenta **20**

CAPÍTULO 1

Cómo leer esta guía

Esta guía está pensada para alguien recién graduado en Ingeniería Informática: sabes programar, conoces estructuras de datos, hilos, HTTP y algo de orientación a objetos, pero **no sabes Rust** y no has tocado QML. Al terminarla deberías poder abrir cualquier archivo del proyecto, entender qué hace y por qué está escrito así, y añadir funciones nuevas sin romper nada.

1.1 El recorrido

Parte I. Empezar con buen pie
Un tutorial de Rust (capítulo 2) que arranca desde cero y se centra en lo que este proyecto usa, con ejercicios que puedes hacer en un proyecto de prácticas aparte. Después, QML (capítulo **??**), el lenguaje de la interfaz. Si ya sabes Rust, salta al capítulo **??**.

Parte II. Cómo está hecho
La arquitectura completa, las herramientas de compilación, el puente entre Rust y QML y, sobre todo, un recorrido *archivo por archivo* por todo el código: el núcleo en Rust, el código nativo en C++ y Objective-C++, cada componente de la interfaz, los formatos de datos y los flujos completos de principio a fin.

Parte III. Por qué está así
Las decisiones de diseño con sus alternativas descartadas, las medidas de rendimiento y memoria que las justifican y las «trampas» que ya nos encontramos (y cómo evitarlas).

Parte IV. Desarrollar por tu cuenta
Recetas paso a paso para los cambios más comunes, cómo depurar, y una colección de ejercicios graduados sobre el propio proyecto.

1.2 Convenciones

- Los nombres de archivo aparecen así: src/player.rs. Las rutas son relativas a la raíz del repositorio.
- El código aparece en bloques. Cuando un bloque es un *extracto* del proyecto se indica de qué archivo sale; a veces está recortado (con `...`) para quedarnos con lo importante. Abre el archivo real al lado: la guía no sustituye al código, lo acompaña.
- Los comandos de terminal van en bloques oscuros:

```
cargo build --release
```

Encontrarás cuatro tipos de recuadro:

Idea clave

Algo que conviene retener: un concepto que aparece por todo el proyecto.

¿Por qué así?

La razón de una decisión: qué alternativa había y por qué se descartó.

Cuidado

Un error que ya cometimos (o que es fácil cometer) y cómo evitarlo.

Ejercicio

Algo para hacer tú. Hacer los ejercicios es la forma más rápida de que esto se quede.

1.3 Documentos hermanos

La guía es el documento largo. En el repositorio hay otros, más cortos, que conviene tener a mano:

README.md Qué es la aplicación y cómo se usa (para usuarios).

COMPILAR.md Compilar e instalar en macOS y CachyOS/Arch.

AGENTS.md La chuleta de quien desarrolla: mapa de archivos, comandos, argumentos de depuración, convenciones y trampas. Si añades algo, añádelo también ahí.

docs/ARCHITECTURE.md Detalle técnico y medidas, en formato corto.

docs/ROADMAP.md Qué está hecho y qué falta.

Esta guía también envejece

El código cambia; si un detalle de la guía no coincide con el archivo, manda el archivo. Cuando cambies algo importante, actualiza el capítulo correspondiente (docs/guia/capitulos/) y vuelve a compilar el PDF.

Parte I

Empezar con buen pie

CAPÍTULO 2

Rust desde cero, lo justo para este proyecto

Rust tiene fama de difícil. Parte de esa fama es merecida: el compilador es estricto y al principio rechaza código que en otros lenguajes funcionaría. La otra parte es que casi todo lo que rechaza son errores reales (accesos a memoria liberada, condiciones de carrera entre hilos) que en C++ o Java descubrirías en producción. Este capítulo te lleva de cero a poder leer y modificar el proyecto. No cubre todo Rust: cubre lo que el proyecto usa, en el orden en que conviene aprenderlo.

2.1 Un proyecto de prácticas

Antes de tocar el reproductor, crea un proyecto aparte para experimentar. Rust se instala con `rustup` (ver COMPILAR.md); después:

```
cargo new practicas
cd practicas
cargo run
```

`cargo` es la herramienta de Rust para todo: crea proyectos, descarga dependencias, compila, ejecuta y pasa las pruebas. `cargo new` crea esta estructura:

```
practicas/├──
 Cargo.toml      ← nombre del proyecto y dependencias («crates»)└──
 src/
    └── main.rs     ← el programa
```

y src/main.rs trae ya el saludo de siempre:

```rust
fn main() {
    println!("Hola, mundo");
}
```

Todos los ejemplos de este capítulo se pueden pegar ahí y ejecutar con `cargo run`. Hazlo: leer Rust no es lo mismo que escribirlo, y el compilador es el mejor profesor que vas a tener.

2.2 Variables, tipos y funciones

```rust
fn main() {
    let titulo = "Columbia";        // inmutable por defecto
    let mut reproducciones = 0;     // `mut` para poder cambiarla
    reproducciones += 1;
```

```
    let duracion: f64 = 186.5;      // tipo explícito (f64 = double)
    let minutos = (duracion / 60.0) as u32;   // conversión explícita con `as`
    println!("{titulo}: {minutos} min, {reproducciones} vez");
}
```

- **Inmutable por defecto.** `let` crea una variable que no se puede cambiar. Si quieres cambiarla, `let mut`. Parece una molestia; en la práctica hace que leer código ajeno sea mucho más fácil, porque sabes qué puede cambiar y qué no.
- **Inferencia de tipos.** Normalmente no escribes el tipo: el compilador lo deduce. Lo escribes cuando quieres dejarlo claro o cuando no puede deducirlo.
- **Sin conversiones implícitas.** Un `f64` no se convierte solo en entero: hay que pedirlo con `as`. Así no hay sorpresas de precisión.

Tipos básicos que verás:

Tipo	Qué es
`i32`, `i64`, `u32`, `u64`, `usize`	enteros con y sin signo (`usize`: índices y tamaños)
`f64`	número decimal de doble precisión
`bool`	`true` o `false`
`char`	un carácter Unicode
`String`	texto propio, que puede crecer
`&str`	texto prestado (una vista de solo lectura sobre un texto)
`Vec<T>`	lista dinámica de elementos de tipo `T` (como `ArrayList<T>`)
`HashMap<K, V>`, `HashSet<T>`	diccionario y conjunto
`(A, B)`	tupla

Las funciones declaran el tipo de sus parámetros y de lo que devuelven:

```
fn segundos(reloj: &str) -> f64 {
    reloj
        .split(':')
        .filter_map(|parte| parte.trim().parse::<f64>().ok())
        .fold(0.0, |total, parte| total * 60.0 + parte)
}
```

Esta es, tal cual, la función `seconds` de src/ytm.rs: convierte `"3:27"` en 207. Fíjate en dos cosas que sorprenden al principio:

1. **No hay `return`.** En Rust casi todo es una expresión que produce un valor, y la *última expresión* de un bloque (sin punto y coma) es lo que el bloque devuelve. `return` existe, pero se usa para salir antes.
2. **Cadenas de métodos.** `split`, `filter_map` y `fold` son *iteradores*: lo verás en la sección 2.9. De momento, léelo como «parte por `:`, convierte cada trozo en número descartando los que no lo sean, y acumula».

Ejercicio

Copia `segundos` a tu proyecto de prácticas y llama a `println!("{}", segundos("1:02:03"))`. ¿Qué devuelve? ¿Y con `"abc"`? Explica por qué.

2.3 Control de flujo

```
let volumen = 120.0_f64;
let volumen = if volumen > 100.0 { 100.0 } else { volumen };  // if es una expresión

for numero in 1..=3 {            // 1, 2, 3 (con `..` sería 1, 2)
    println!("pista {numero}");
}

let mut intentos = 0;
while intentos < 2 {
    intentos += 1;
}

let resultado = loop {           // bucle infinito que devuelve un valor con break
    intentos += 1;
    if intentos == 5 { break intentos * 10; }
};
```

Fíjate en el segundo `let volumen`: en Rust puedes «tapar» (*shadowing*) una variable declarando otra con el mismo nombre. El proyecto lo usa a menudo para ir refinando un valor sin inventar nombres (`let artist = ...; let artist = if ...`).

2.4 Structs: datos con nombre

Un `struct` agrupa datos. Es lo más parecido a una clase, pero sin herencia. Esta es la pista de reproducción del proyecto (src/ytm.rs):

```
#[derive(Clone, Debug, Default, Serialize, Deserialize)]
#[serde(rename_all = "camelCase", default)]
pub struct Track {
    pub title: String,
    pub artist: String,
    pub thumb: String,
    pub video_id: String,
    pub duration: f64,
}
```

- `pub` hace visible el struct (o un campo) desde otros archivos. Sin `pub`, es privado del archivo (del «módulo»).
- Lo que empieza por `#[...]` son *atributos*: instrucciones para el compilador o para una biblioteca. `#[derive(...)]` le pide que *genere código* automáticamente:
  - `Clone`: poder duplicar el valor con `.clone()`.

- `Debug`: poder imprimirlo con `println!("{:?}", pista)`.
- `Default`: un valor «vacío» (`Track::default()`: textos vacíos, duración 0).
- `Serialize`, `Deserialize`: convertir a JSON y desde JSON, con la biblioteca `serde`. La segunda línea, `rename_all = "camelCase"`, hace que `video_id` se escriba `videoId` en el JSON, que es lo que espera la interfaz.

Crear y usar un struct:

```
let pista = Track {
    title: "Columbia".to_owned(),     // &str → String
    artist: "Quevedo".to_owned(),
    video_id: "abc123".to_owned(),
    ..Track::default()                // el resto, con sus valores por defecto
};
println!("{} — {}", pista.title, pista.artist);
```

`"Columbia"` es un `&str` (un texto fijo dentro del programa) y el campo es un `String` (un texto propio). `.to_owned()` (o `String::from`, o `.to_string()`) hace la copia. Verás muchísimos `to_owned()`: es normal.

2.4.1 Métodos

Los métodos se escriben en un bloque `impl`. El primer parámetro dice cómo recibe el método a su propio objeto:

```
impl Track {
    fn minutos(&self) -> f64 {            // &self: lo lee, no lo cambia
        self.duration / 60.0
    }
    fn renombrar(&mut self, nuevo: &str) { // &mut self: lo modifica
        self.title = nuevo.to_owned();
    }
    fn nueva(titulo: &str) -> Track {      // sin self: «función estática», se llama
    Track::nueva(...)
        Track { title: titulo.to_owned(), ..Track::default() }
    }
}
```

2.5 Enums y `match`

Un `enum` de Rust es mucho más que una lista de constantes: **cada variante puede llevar sus propios datos**. Son ideales para representar «una cosa que puede ser de varias formas». Los eventos que el motor de audio manda al reproductor (src/audio.rs):

```
pub enum Event {
    Codec(String),      // lleva el nombre del códec
    Bitrate(f64),       // lleva los kbps
    Position(f64),
    Duration(f64),
    Paused(bool),
    Started,            // no lleva nada
```

```
    Ended,
    Failed,
}
```

Se tratan con `match`, que compara el valor con cada forma posible y **obliga a cubrir todos los casos** (si añades una variante y olvidas tratarla, no compila):

```
match event {
    Event::Position(position) => self.set_position(position),
    Event::Duration(duration) if duration > 0.0 => self.set_duration(duration),
    Event::Duration(_) => {}                    // `_`: no me importa el valor
    Event::Started => { /* ... */ }
    Event::Ended => { /* pasar a la siguiente */ }
    _ => {}                                     // cualquier otro caso
}
```

Es un extracto simplificado de `Player::on_event` en src/player.rs. La línea con `if` es una *guarda*: esa rama solo vale si se cumple la condición.

Idea clave

`match` es una de las herramientas más usadas en Rust. Cuando veas uno, léelo como un `switch` que además «desempaqueta» los datos de cada variante en variables.

2.6 Sin null ni excepciones: `Option` y `Result`

Rust no tiene `null`. Tampoco excepciones. Para ambos casos usa dos enums de la biblioteca estándar:

```
enum Option<T> { Some(T), None }        // un valor que puede no estar
enum Result<T, E> { Ok(T), Err(E) }     // una operación que puede fallar
```

(La `T` y la `E` son *genéricos*: `Option<String>` es «quizá un texto», `Result<Vec<Track>, String>` es «una lista de pistas o un mensaje de error».)

Ejemplos reales:

```
// src/account.rs: puede que no haya sesión guardada.
pub fn cookies_file() -> Option<PathBuf> { ... }

// src/stream.rs: la URL del audio, o por qué no se pudo.
pub fn resolve(video_id: &str) -> Result<String, String> { ... }
```

Como son enums, se pueden tratar con `match`, pero hay formas más cortas que el proyecto usa constantemente:

```
// 1) if let: «si es Some (u Ok), dame el valor»
if let Some(url) = resolver::cached(&video_id) {
    self.start_stream(Some(&url), &video_id);
}

// 2) let ... else: «si no es Some, sal»
```

```rust
let Some(track) = self.tracks.get(index).cloned() else { return };

// 3) valores por defecto
let nombre = info.name.unwrap_or_default();      // "" si no había
let volumen = guardado.unwrap_or(100.0);

// 4) transformar sin desempaquetar
let mayusculas = titulo_opcional.map(|t| t.to_uppercase());
```

2.6.1 El operador ?

El más importante: ? significa «si esto es un error (o None), devuélvelo inmediatamente desde la función actual; si no, dame el valor de dentro».

```rust
// src/ytm.rs (simplificado)
pub fn browse(browse_id: &str) -> Result<Vec<Zone>> {
    let pagina = post("browse", json!({ "browseId": browse_id }))?;  // si falla la red,
     sale con el error
    Ok(zones(&pagina))
}
```

Sin ? habría que escribir un `match` en cada llamada que puede fallar. Con él, el camino feliz se lee de corrido y los errores suben solos hasta quien sepa qué hacer con ellos (normalmente, mostrar un mensaje en la interfaz).

unwrap()

`.unwrap()` saca el valor de un `Option`/`Result` y, si no lo hay, **cierra el programa de golpe** («panic»). En este proyecto no se usa en código normal: un fallo de red no debe tumbar el reproductor. Solo aparece en las pruebas, donde un fallo debe parar la prueba.

Ejercicio

Escribe `fn primera_palabra(texto: &str) -> Option<&str>` que devuelva la primera palabra o None si el texto está vacío. Pista: `texto.split_whitespace().next()`. Después escribe `fn doble(texto: &str) -> Result<i32, String>` que convierta el texto en número y lo multiplique por dos, usando ? y `map_err` para convertir el error en texto.

2.7 Propiedad y préstamos

Aquí está lo que hace a Rust distinto. Rust no tiene recolector de basura (como Java) ni te obliga a liberar memoria a mano (como C). En su lugar sigue tres reglas que el compilador comprueba:

1. **Cada valor tiene un único dueño** (una variable).
2. **Cuando el dueño desaparece** (sale de su bloque), **el valor se libera**.
3. Puedes **prestar** el valor: muchos préstamos de solo lectura (`&`) *o* un único préstamo para modificar (`&mut`), nunca las dos cosas a la vez.

2.7.1 Mover

```
let a = String::from("Columbia");
let b = a;              // el texto se MUEVE a `b`: ahora `b` es el dueño
println!("{a}");        // ERROR: `a` ya no tiene nada
```

No se copió el texto: se pasó la propiedad. Esto evita que dos variables liberen el mismo texto. Lo mismo pasa al llamar a una función que recibe un `String`: se lo das y ya no es tuyo.

2.7.2 Prestar

```
fn imprime(texto: &str) { println!("{texto}"); }       // pide prestado
fn alarga(texto: &mut String) { texto.push('!'); }      // pide prestado para modificar

let mut titulo = String::from("Columbia");
imprime(&titulo);        // préstamo de lectura: `titulo` sigue siendo tuyo
alarga(&mut titulo);     // préstamo de escritura
imprime(&titulo);        // "Columbia!"
```

2.7.3 Clonar

Si necesitas tu propia copia, `.clone()`. En este proyecto los datos (textos, listas de unas decenas de pistas) son pequeños, y clonar es la solución habitual cuando el compilador protesta. Es barato y claro.

```
let pistas = self.tracks.clone();   // copia la lista para usarla en otro hilo
```

2.7.4 `Copy`

Los tipos pequeños (números, `bool`, `char`) se *copian* en vez de moverse: tras `let b = a;` puedes seguir usando `a`. Por eso nunca tendrás problemas de propiedad con un `f64`.

2.7.5 Cómo resolver los errores de propiedad

Los dos errores que más verás:

```
error[E0382]: borrow of moved value: `titulo`
error[E0502]: cannot borrow `x` as mutable because it is also borrowed as immutable
```

Casi siempre se arreglan con una de estas:

1. Prestar (`&titulo`) en vez de mover (`titulo`).
2. `.clone()` si de verdad necesitas dos copias.
3. Reordenar: usar el valor *antes* de moverlo, o terminar de leer antes de empezar a modificar.
4. Guardar en una variable lo que necesitas antes de pedir el préstamo mutable:

```
let duracion = *self.duration();        // copio el número
self.as_mut().set_position(duracion);   // ahora ya puedo modificar
```

Idea clave

Lee el mensaje de error *entero*. El compilador de Rust señala la línea donde se movió el valor, la línea donde se usó después, y a menudo sugiere el arreglo exacto (*help: consider cloning the value*). `rustc --explain E0382` da la explicación larga.

2.7.6 `String` frente a `&str`

Ahora se entiende la diferencia: un `String` es un texto *propio* (lo puedes guardar en un struct, modificar, devolver); un `&str` es un *préstamo* de un texto que es de otro. Regla práctica: **los parámetros de funciones, `&str`; los campos de structs y lo que devuelves, `String`**.

2.7.7 Vidas (*lifetimes*), solo para reconocerlas

A veces verás `'a` en una firma:

```
fn find<'a>(value: &'a Value, key: &str) -> Option<&'a Value> { ... }   // src/ytm.rs
```

Se lee: «lo que devuelvo es un préstamo de `value`, así que vive lo mismo que él». Es una anotación para el compilador, no cambia lo que hace la función. En este proyecto aparece muy poco; si escribes una función que devuelve un préstamo y el compilador te pide una, copia este patrón.

Ejercicio

En tu proyecto de prácticas, haz que compile esto sin cambiar la función `guarda`:

```
fn guarda(lista: &mut Vec<String>, texto: String) { lista.push(texto); }
fn main() {
    let mut lista = Vec::new();
    let titulo = String::from("Halo");
    guarda(&mut lista, titulo);
    println!("{titulo} guardado; hay {}", lista.len());
}
```

Hay al menos dos soluciones. ¿Cuál gasta más memoria?

2.8 Colecciones

```
let mut pistas: Vec<Track> = Vec::new();
pistas.push(pista);
let primera = pistas.first();          // Option<&Track>
let tercera = pistas.get(2);           // Option<&Track> (no se sale del vector)
let n = pistas.len();
pistas.remove(0);
```

```rust
use std::collections::HashMap;
let mut colas: HashMap<String, Vec<Track>> = HashMap::new();
colas.insert("OLAK5uy...".to_owned(), pistas);
if let Some(cola) = colas.get("OLAK5uy...") { /* ... */ }
```

`vec[i]` también existe, pero si `i` se sale del vector el programa se cierra (panic). Por eso el proyecto usa casi siempre `.get(i)`, que devuelve `Option`.

2.9 Clausuras e iteradores

Una *clausura* es una función anónima (una lambda): `|parámetros| cuerpo`.

```rust
let doble = |n: i32| n * 2;
let sumar = |a, b| { let s = a + b; s };
```

Los *iteradores* encadenan operaciones sobre colecciones. Son perezosos: no hacen nada hasta que algo consume el resultado (`collect`, `sum`, `for`...).

```rust
// Nombres de las pistas de más de 3 minutos, en mayúsculas
let nombres: Vec<String> = pistas
    .iter()                                  // recorre por referencia
    .filter(|p| p.duration > 180.0)          // se queda con algunas
    .map(|p| p.title.to_uppercase())         // transforma cada una
    .collect();                              // junta el resultado en un Vec

let total: f64 = pistas.iter().map(|p| p.duration).sum();
let hay_larga = pistas.iter().any(|p| p.duration > 600.0);
let posicion = pistas.iter().position(|p| p.video_id == actual);   // Option<usize>
```

Operaciones que verás en el proyecto: `iter`, `map`, `filter`, `filter_map` (transforma y descarta los `None`), `find`, `position`, `any`, `take(n)`, `flatten`, `collect`, `enumerate` (da también el índice), `rev`, `fold`.

2.9.1 `move` en clausuras

Una clausura puede usar variables de fuera. Por defecto las *toma prestadas*. Con `move` se *las queda* (las mueve dentro). Es imprescindible cuando la clausura va a vivir más que la función que la crea, por ejemplo al mandarla a otro hilo:

```rust
let video_id = track.video_id.clone();
thread::spawn(move || {            // el hilo se queda con su copia de video_id
    let url = resolver::resolve(&video_id);
});
```

2.10 Traits: comportamientos compartidos

Un *trait* declara un comportamiento que varios tipos pueden tener, como una interfaz de Java. En este proyecto casi nunca escribirás uno, pero usarás muchos:

- `Default`, `Clone`, `Debug`: los generas con `#[derive]`.
- `Serialize`/`Deserialize` de serde: también con `#[derive]`.
- `Drop`: código que se ejecuta cuando un valor se libera. El motor de audio lo usa para cerrar mpv al destruir el objeto (src/audio.rs):

```rust
impl Drop for Mpv {
    fn drop(&mut self) {
        let _ = self.child.kill();      // mata el proceso de mpv
    }
}
```

Y verás parámetros como `impl Fn(Event) + Send + 'static`: «cualquier función que reciba un `Event`, que se pueda mandar a otro hilo (`Send`) y que no tome prestado nada temporal (`'static`)». Es la firma de `Mpv::spawn`, que recibe la función a la que avisar de cada evento.

2.11 Módulos y visibilidad

Cada archivo src/x.rs es un *módulo*. src/main.rs los declara:

```rust
mod account;
mod audio;
mod player;
// ...
```

Desde otro archivo se usan con su ruta (`crate::audio::which(...)`) o importándolos:

```rust
use crate::audio::{search_path, which};       // ahora basta con which(...)
use std::collections::HashMap;                 // de la biblioteca estándar
use serde_json::{json, Value};                 // de una crate externa
```

Solo es visible desde fuera lo marcado con `pub`. Un módulo puede tener submódulos: los plugins están en src/plugins.rs y declaran `mod discord;` y `mod lastfm;`, que viven en src/plugins/discord.rs y src/plugins/lastfm.rs.

2.12 Macros

Lo que acaba en `!` es una *macro*: código que genera código al compilar.

`println!`, `eprintln!`	escribir en la salida estándar o en la de errores
`format!`	como `println!` pero devuelve un `String`
`vec![a, b]`	crear un `Vec`
`json!({...})`	escribir JSON literal (de `serde_json`)
`include_str!("x")`	meter el contenido de un archivo dentro del binario al compilar
`assert!`, `assert_eq!`	comprobaciones en las pruebas
`matches!(x, patrón)`	`true` si `x` encaja con el patrón

En los textos de `format!/println!`, `{nombre}` inserta una variable, `{}` inserta el siguiente argumento y `{:?}` usa la forma de depuración (`Debug`), útil para imprimir casi cualquier cosa.

2.13 Hilos y estado compartido

El proyecto hace mucho trabajo lento (peticiones a YouTube, arrancar yt-dlp, hablar con mpv) y **la interfaz no puede quedarse congelada**. Por eso lo lento siempre va en otro hilo.

2.13.1 Lanzar un hilo

```
use std::thread;

let handle = thread::spawn(move || {
    // trabajo lento
    42
});
let resultado = handle.join();   // esperar al hilo (el proyecto casi nunca espera)
```

2.13.2 Compartir datos: `Mutex`

Si dos hilos tocan el mismo dato a la vez, hay una condición de carrera. Rust *no compila* ese código. Para compartir datos se usa un `Mutex` (un candado): para tocar el dato hay que coger el candado, y solo un hilo puede tenerlo.

```
use std::sync::{LazyLock, Mutex};
use std::collections::HashMap;

// Una variable global, creada la primera vez que se usa (LazyLock), protegida por un
    Mutex.
static QUEUES: LazyLock<Mutex<HashMap<String, Vec<Track>>>> = LazyLock::new(Default::
    default);

fn cached_queue(playlist_id: &str) -> Option<Vec<Track>> {
    QUEUES.lock().ok()?.get(playlist_id).cloned()
    //     ^ coge el candado (puede fallar si otro hilo se cayó con él: por eso .ok()?)
}   // aquí se suelta el candado solo, al salir del bloque
```

Es, tal cual, la caché de colas de src/player.rs. El candado se suelta solo cuando la variable que lo tiene desaparece: no hay `unlock`.

2.13.3 Otros mecanismos que verás

`AtomicBool`, `AtomicU64` Un número o booleano que varios hilos pueden leer y escribir sin candado. Para indicadores simples (`PREMIUM` en src/resolver.rs).

`Condvar` «Esperar hasta que otro hilo avise». El resolvedor lo usa para que, si dos partes del programa piden la misma URL a la vez, la segunda espere a la primera en vez de lanzar otro yt-dlp.

`mpsc::channel` Una cola de mensajes entre hilos: uno envía (`Sender`), otro recibe (`Receiver`). Los plugins la usan: el reproductor manda eventos y un hilo propio los reparte a Discord y Last.fm.

2.13.4 El patrón del proyecto: trabajar en otro hilo y volver al de Qt

Qt exige que todo lo que toca la interfaz se haga *en su hilo*. Así que el patrón, que encontrarás en casi todos los archivos, es: lanzar un hilo para el trabajo lento y, al terminar, *encolar* una función que se ejecutará en el hilo de Qt.

```
// src/library.rs, búsqueda (simplificado)
let qt = self.qt_thread();                    // «buzón» del hilo de Qt
thread::spawn(move || {
    let result = ytm::search(&query);         // lento: red
    let _ = qt.queue(move |mut library| {     // esto se ejecuta luego en el hilo de Qt
        match result {
            Ok(zones) => library.as_mut().set_busqueda(QString::from(&to_json(&zones))),
            Err(error) => library.as_mut().set_error(QString::from(&error)),
        }
    });
});
```

Lo verás con detalle en el capítulo **??**. Quédate con la idea: **`thread::spawn` para lo lento, `qt.queue` para volver**.

2.14 Llamar a C y C++: `extern` y `unsafe`

Unas pocas partes del proyecto están en C++ u Objective-C++ (lo que Qt no expone a QML y la ventana de inicio de sesión de macOS). Rust las llama así (src/main.rs):

```
extern "C" {
    fn musicplayer_tune_windows();          // definida en src/window.cpp
}

// SAFETY: función C++ sin argumentos; se llama en el hilo de Qt.
unsafe { musicplayer_tune_windows() };
```

`unsafe` no significa «peligroso», sino «aquí el compilador no puede comprobarlo; lo garantizo yo». Por convención, cada `unsafe` lleva encima un comentario `SAFETY:` explicando por qué es correcto. En el proyecto solo se usa para estas llamadas.

2.15 Atributos condicionales y pruebas

```
#[cfg(target_os = "macos")]          // esta función solo existe al compilar para macOS
fn abrir_ajustes() { ... }

if cfg!(target_os = "macos") { ... } // lo mismo, como condición en una expresión
```

Las pruebas viven al final de cada archivo, en un módulo que solo se compila al probar:

```rust
#[cfg(test)]
mod tests {
    use super::*;                    // todo lo del archivo

    #[test]
    fn el_artista_sale_limpio_del_subtitulo() {
        assert_eq!(artist_from_subtitle("Quevedo • 75 M reproducciones"), "Quevedo");
    }
}
```

```
cargo test --release
```

2.16 Leer los errores del compilador

Un ejemplo completo y cómo leerlo:

```
error[E0308]: mismatched types
  --> src/player.rs:312:23
   |
312 |         self.load(next);
   |              ---- ^^^^ expected `usize`, found `i32`
   |              |
   |              arguments to this method are incorrect
   |
help: you can convert an `i32` to a `usize` and panic if the converted value doesn't fit
   |
312 |         self.load(next.try_into().unwrap());
```

1. La primera línea dice *qué* pasa (tipos que no coinciden) y su código (E0308).

2. La flecha dice *dónde*: archivo, línea y columna.

3. El dibujo señala exactamente la expresión y explica (se esperaba `usize`, llegó `i32`).

4. *help* propone un arreglo. No siempre es el mejor (aquí, `next as usize` es más simple), pero apunta a la causa.

2.17 Resumen y siguientes pasos

Ves esto	Significa
`let mut x`	variable modificable
`&x`, `&mut x`	préstamo de lectura / de escritura
`x.clone()`	copia propia
`Option<T>`, `Some`, `None`	valor que puede faltar
`Result<T, E>`, `Ok`, `Err`	operación que puede fallar
`x?`	si es error/None, sal de la función con él
`if let Some(v) = x`	si hay valor, úsalo
`let Some(v) = x else { return }`	si no hay valor, sal
`match`	elegir según la forma del valor
`|a| ...`, `move ||`	clausura; que se queda con lo que usa
`#[derive(...)]`	generar código automáticamente
`x!(...)`	macro
`thread::spawn`, `Mutex`, `LazyLock`	hilos y estado compartido
`unsafe`, `extern "C"`	llamar a código C/C++

Para ir más allá (en este orden):

1. *The Rust Programming Language* («el libro», gratis en `https://doc.rust-lang.org/book/`): capítulos 1 a 10, 13 (iteradores y clausuras) y 16 (concurrencia).
2. *Rust by Example* (`https://doc.rust-lang.org/rust-by-example/`): ejemplos cortos de cada tema.
3. *Rustlings* (`https://github.com/rust-lang/rustlings`): ejercicios pequeños que se corrigen solos.

Ejercicio de cierre

En tu proyecto de prácticas, escribe un programa que:

1. Tenga un `struct Pista { titulo: String, segundos: f64 }`.
2. Cree un `Vec<Pista>` con cinco pistas.
3. Lance un hilo que calcule la duración total y la devuelva.
4. Imprima la pista más larga (pista: `iter().max_by(|a, b| a.segundos.total_cmp(&b.segundos))`).

Si compila a la primera, enhorabuena; si no, lee cada error con calma: cada uno te enseña algo.

Parte II

Cómo está hecho

Parte III

Por qué está así

Parte IV

Desarrollar por tu cuenta